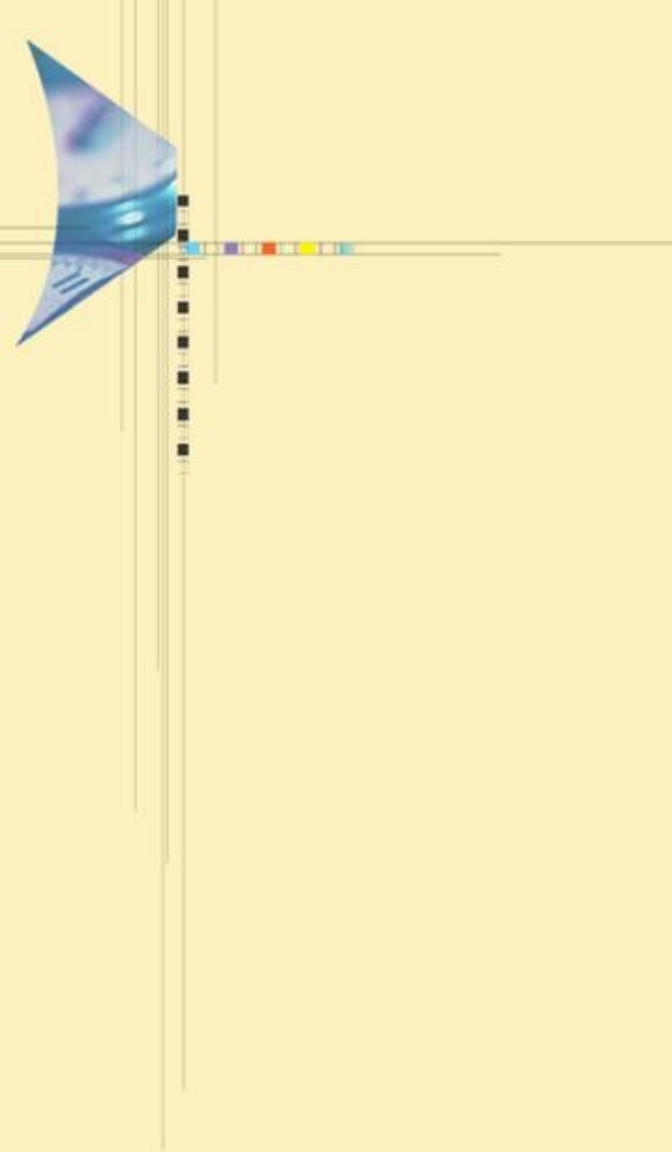




第二讲

归纳法

主要内容



2.1 引言

2.2 归纳法策略的算法设计思想及分析方法

2.3. 归纳策略的经典应用（可选讲部分）



主要内容



2.1 引言

2.1.1 简单问题例的算法描述（插入排序、选择排序）

2.1.2 简单问题例算法的分析

2.1 引言

- 递归最简单的描述是将一个问题分割成一个或多个的子问题，这些子问题与原问题具有相同的结构，然后组合这些子问题的解决方案，以获得原来的问题的解决方案的过程。

例 1 阶乘函数

- 阶乘函数可递归地定义为

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

边界条件

递归方程

- 边界条件**与**递归方程**是递归函数的二个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。

例 2 Fibonacci 数列

- 无穷数列 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ..., 被称为 Fibonacci 数列。它可以递归地定义为:

$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

边界条件

递归方程

Fibonacci 数列递归算法

第 n 个 Fibonacci 数可递归地计算如下：

```
int fibonacci(int n)
{
    if (n <= 1) return 1;
    return fibonacci(n-1)+fibonacci(n-2);
}
```

2.1.1 简单问题例的算法描述

- 先用两个简单的例子来展示归纳法技术的本质。
 - Selection sort （选择排序）
 - Insertion sort （插入排序）



2.1.1 Selection sort (选择排序)



基本思想

令 $A[1..n]$ 作为算法的输入，由归纳法假设我们知道如何对后 $n-1$ 个元素排序，即对 $A[2..n]$ 排序。对某个 j ， $j, 1 \leq j \leq n$ ，设 $A[j]$ 是最小值。

- 首先，如果 $j \neq 1$ ，我们交换 $A[1]$ 和 $A[j]$
- 然后由归纳法假设，我们已知如何对 $A[2..n]$ 排序，因此可对在 $A[2..n]$ 中的元素递归的排序



算法 2.1 SELECTIONSORTREC

输入：n 个元素的数组 $A[1...n]$ 。

输出：按非降序排列的数组 $A[1...n]$

1.sort(1)

过程 sort(i) { 对 $A[i...n]$ 排序 }

1. if $i < n$ then

2. $k \leftarrow i$

3. for $j \leftarrow i+1$ to n

4. if $A[j] < A[k]$ then $k \leftarrow j$ // 找到后面的最小元素

2. end for

6. if $k \neq i$ then 互换 $A[i]$ and $A[k]$

7. sort($i+1$)

8. end if

时间复杂度

- 令 $C(n)$ 表示元素比较的次数
 - 显然, $C(1) = 0$.
 - 过程 sort 第一次被调用时 $i = 1$, 数组中有 n 个元素, 因此在这次调用中, 元素比较的总次数由两部分元素比较次数之和组成, 即 $n-1$ 次元素比较次数加上对 $A[2..n]$ 排序比较次数, 后一比较次数是所有对 $\text{sort}(i+1)$ 的递归调用的总和。而从递归调用得到的元素比较次数是 $C(n-1)$ 。把这些放在一起, 有:

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + (n-1) & \text{if } n \geq 2 \end{cases}$$

这个递推式的解是

$$C(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2}$$

- 由于运行时间和元素的比较次数成线性关系, 所以说它是 $O(n^2)$ 的。

2.1.2 Insertion sort (插入排序)



基本思想

- 假设我们知道如何对前 $n-1$ 个元素，也就是数组 $A[1..n-1]$ 排序。
- 那么在对 $A[1..n-1]$ 排序后，我们只要把 $A[n]$ 插入它的适当位置，假设是位置 j ， $1 \leq j \leq n$ 。
- 这个插入可能导致 $A[j+1]$, $A[j+2]$, ..., $A[n-1]$ 移到位置 $j+2$, $j+3$, ..., n 。



算法 2.1.2 INSERTIONSORTREC

输入：n 个元素的数组 $A[1...n]$

输出：按非降序排列的数组 $A[1...n]$

1. $sort(n)$

流程 $sort(i)$ { 对 $A[1...i]$ 排序 }

1. **if** $i > 1$ **then**

2. $x \leftarrow A[i]$

3. $sort(i-1)$

4. $j \leftarrow i-1$

2. **while** $j > 0$ **and** $A[j] > x$ // 如果当前值大于 x ，且不到第一个元素

6. $A[j+1] \leftarrow A[j]$ // 当前值后移一位

7. $j \leftarrow j-1$

8. **end while**

9. $A[j+1] \leftarrow x$

10. **end if**

算法分析

- $N=1$, 显然 $C(1)=0$;
- $N=n-1$ 时的代价设为 $C(n-1)$
- $N=n$ 时, 代价两部分: 一部分是 $C(n-1)$, 另一部分是有 $N=n-1$ 的解归纳出 $N=n$ 的解
 - 最好情况, 当前值大于 $A[j]>x$

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + 1 & \text{if } n \geq 2 \end{cases} \Rightarrow C(n) = n - 1$$

- 最坏情况, 需要 $n-1$ 次比较 (逆序)

$$C(n) = \begin{cases} 0 & \text{if } n = 1 \\ C(n-1) + (n-1) & \text{if } n \geq 2 \end{cases} \Rightarrow C(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2}$$

主要内容



2.2 归纳法策略的算法设计思想及分析方法

2.2.1 归纳算法框架

2.2.2 归纳算法的一般分析方法



2.2.1 归纳算法框架

- 数学方面
 - 考虑一个带有参数 n 的问题，在问题的实例中 n 通常表示事物的数目。
 - 当我们寻找这类问题的解时，有时候从求解一个带有小一点参数的相同问题开始，例如参数是 $n-1, n/2$ 等，然后再把解推广到包含所有的 n 个物体。
 - 这样，问题的解决会比较容易些。这种方法基于众所周知的数学归纳法证明技术。

2.2.2 归纳算法的一般分析方法



递归思想

- 从本质上说，给出一个带有参数 n 的问题，用归纳法设计一个算法是基于这样一个事实：如果我们知道如何求解带有参数小于 n 的同样问题（它被称为归纳假设），那么我们的任务就化为如何把解法扩展到带有参数 n 的实例。
- 这种设计技术的一个优点是（一般地，对于所有的递归算法来说都是如此）：它所设计出的算法的正确性证明已自然地嵌入了算法的描述中了。



主要内容



2.3. 归纳策略的经典应用（可选讲部分）

2.3.1 求幂问题的算法

2.3.2 多项式求值问题

2.3.3 全排列问题

2.3.4 基数排序问题

2.3.5 多数元素问题

2.3.6 汉诺塔问题

2.3.7 取棋子游戏问题



2.3.1 Integer exponentiation (整数幂)

? 问题描述

- 对求实数 x 的 n 次幂设计一个有效的算法
- 直接的方法
 - 对 x 用迭代方法自乘 n 次，这种方法是十分低效的，因为它需要 $\Theta(n)$ 乘法。按输入的大小来说，它是指数的 (见 1.14 节)

? Can we find much more efficient method?

一种有效的方法



基本思路

- 令 $m = \lfloor n/2 \rfloor$ ，假设已经知道如何计算 x^m ，那么有两种情况：

- 如果 n 是偶数，那么

$$x^n = (x^m)^2$$

- 否则，

$$x^n = x(x^m)^2$$



算法 2.3.1-1 EXPREC

Input: 实数 x 和非负整数 n 。

Output: x^n .

1. power(x , n)

流程 power(x , m) { 计算 x^m }

1. **if** $m=0$ **then** $y \leftarrow 1$

2. **else**

3. $y \leftarrow \text{power}(x, \lfloor m/2 \rfloor)$

4. $y \leftarrow y^2$

2. **if** m 是奇数 **then** $y \leftarrow xy$

6. **end if**

7. **return** y



算法 2.3.1-2 EXP

Input: 实数 x 和非负整数 n 。

Output: x^n .

1. $y \leftarrow 1$
2. 将 n 用二进制 $d_k d_{k-1} \dots d_0$ 表示
3. **for** $j \leftarrow k$ **downto** 0
4. $y \leftarrow y^2$
2. **if** $d_j = 1$ **then** $y \leftarrow xy$
6. **end for**
7. **return** y

p94





算法分析

- 假定每次乘法的时间是常数，那么这两个版本的算法的运行时间是 $O(\log n)$ ，它对于输入的大小来说是线性的。

2.3.2 多项式求值 (Horner's 规则)

? 问题描述：

- 假设有 $n+2$ 个实数 $a_0, a_1, \dots, a_n$ 和 x 的序列，要对如下多项式求值。

$$P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

- 直接的方法是对每一项分别求值。
- 这种方法是十分低效的，因为它需要 $n+n-1+\dots+1=n(n+1)/2$ 次乘法。

更快的方法 (Horner's rule)

- 基本思想

首先观察

$$\begin{aligned} p_n(x) &= a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 \\ &= (((\dots((a_n x + a_{n-1})x + a_{n-2})x + a_{n-3})\dots)x + a_1)x + a_0 \end{aligned}$$

这种求值的安排称为 **Horner 规则**。

导出：

$$p_{n-1}(x) = a_n x^{n-1} + a_{n-1} x^{n-2} + \dots + a_2 x + a_1$$

得到以下更有效的方法

$$p_n(x) = x p_{n-1}(x) + a_0, \quad p_j(x) = x p_{j-1}(x) + a_{n-j}$$

算法 2.3.2 HORNER

Input: $n+2$ 个实数 $a_0, a_1, \dots, a_n$ x 的序列

Output: $P_n(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$

1. $p \leftarrow a_n$
2. **for** $j \leftarrow 1$ **to** n
3. $p \leftarrow xp + a_{n-j}$
4. **end for**
2. **return** p



算法分析

- 算法 HORNER 的复杂度代价是 n 次乘法和 n 次加法，
- 这是利用归纳假设的优点得到的令人瞩目的成绩。

2.3.3 Generating permutations (生成排列)

- 问题描述

- 研究对于数 $1, 2 \dots n$ 生成所有的排列问题，基于一个假设：我们可以生成 $n-1$ 个数的所有排列。



2.3.3 Generating permutations (生成排列)

- 要构造一个简单生成集合 $\{1,2,3,\dots,n\}$ 的所有排列的算法。观察发现：
 - 如果整数 n 从 $\{1,2,3,\dots,n\}$ 的一个排列中删除，那么结果则是 $\{1,2,3,\dots,n-1\}$ 的一个排列。

2.3.3.1 第一种算法

- 基本思想

假定可以生成 $n-1$ 个数的所有排列

- 生成数 $2, 3, \dots, n$ 的所有排列，并且在每个排列的前面加上数 1
- 接下来，生成数 $1, 3, 4, \dots, n$ 的所有排列，并且在每个排列的前面加上数 2。
- 重复这个过程直到最后生成 $1, 2, \dots, n-1$ 的所有排列，并且在每个排列的前面加上数 n 。

算法 2.3.3.1 PERMUTATIONS1

Input: 正整数 n .

Output: 数 $1, 2, \dots, n$. 的所有可能排列

1. **for** $j \leftarrow 1$ **to** n // 生成数组
2. $P[j] \leftarrow j$
3. **end for**
4. $\text{perm1}(1)$

过程 $\text{perm1}(m)$

1. **if** $m=n$ **then output** $P[1 \dots n]$
2. **else**
3. **for** $j \leftarrow m$ **to** n
4. 互换 $P[j]$ and $P[m]$
5. $\text{perm1}(m+1)$
6. 互换 $P[j]$ and $P[m]$
7. **comment:** at this point $P[m \dots n] = m, m+1, \dots, n$
8. **end for**
9. **end if**

算法分析

- 由于存在有 $n!$ 个排列，过程 perm1 的第 1 步共有 $nn!$ 次来
- 输出所有的排列。
- 现在计算 for 循环的循环次数。
 - 在第一次调用过程 perm1 时， $m = 1$ ，因此 for 循环执行 n 次加上递归调用 perm1(2) 的执行次数。
 - 由于当 $n=1$ 时反复次数是 0 ，因此反复次数 $f(n)$ 能够用以下递推式表示

$$f(n) = \begin{cases} 0 & \text{if } n=1 \\ nf(n-1)+n & \text{if } n \geq 2 \end{cases}$$

- (书本 p96)

算法分析 (继续)

令 $n!h(n)=f(n)$ (注意 $h(1)=0$). (书本 p96)

那么 $n!h(n) = n(n-1)!h(n-1) + n$

这个递推式的解是

$$h(n) = h(1) + \sum_{j=2}^n \frac{1}{(j-1)!} = \sum_{j=1}^{n-1} \frac{1}{j!} < \sum_{j=1}^{\infty} \frac{1}{j!} = (e-1)$$

这里 $e=2.7182818\dots$ 因此,

$$f(n) = n!h(n) = n! \sum_{j=1}^{n-1} \frac{1}{j!} < 2n!$$

- 所以, 算法的运行时间由输出语句来决定, 也就是 $\square (nn!)$ 。

2.3.3.2 第二种算法

- 基本思想

- 首先把 n 放入 $P[1]$, 并且在 $P[2..n]$ 子数组中生成前 $n-1$ 个数的所有排列。
- 接下来把 n 放入 $P[2]$, 并且在子数组 $P[1]$ 和 $P[3..n]$ 中生成前 $n-1$ 个数的所有排列。
- 然后把 n 放入 $P[3]$, 并且在子数组 $P[1,2]$ 和 $P[4..n-1]$ 中生成前 $n-1$ 个数的所有排列。
- 这样继续下去, 直到最后把 n 放到 $P[n]$ 中, 并且在子数组 $P[1..n-1]$ 中生成 $n-1$ 个数的所有排列。
- 初始的时候, $P[1..n]$ 的 n 个项为 0 。

算法 2.3.3.2 PERMUTATIONS2

Input: 正整数 n 。

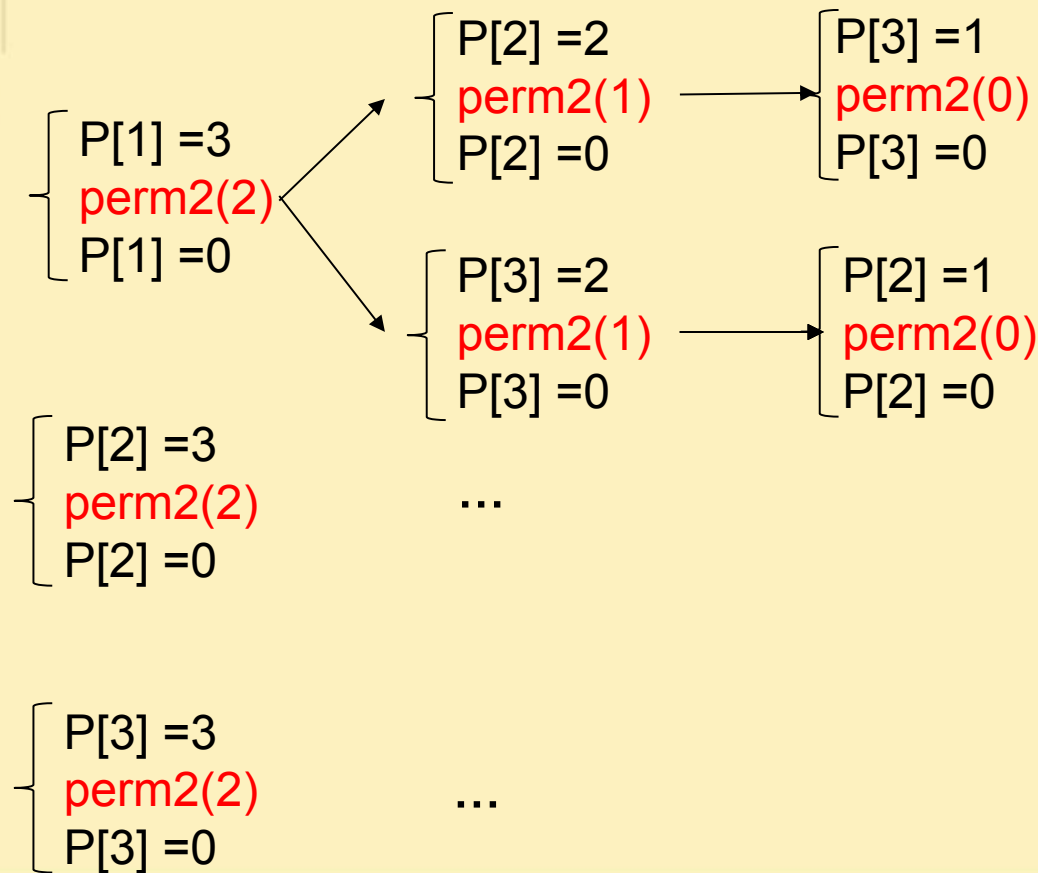
Output: 数 $1, 2, \dots, n$ 的所有可能排列。

1. **for** $j \leftarrow 1$ **to** n
2. $P[j] \leftarrow 0$
3. **end for**
4. $\text{perm2}(n)$

过程 $\text{perm2}(m)$

1. **if** $m=0$ **then output** $P[1..n]$
2. **else**
3. **for** $j \leftarrow 1$ **to** n
4. **if** $P[j]=0$ **then**
5. $P[j] \leftarrow m$
6. $\text{perm2}(m-1)$
7. $P[j] \leftarrow 0$
8. **end if**
9. **end for**
10. **end if**

初始 : $P[1,2,3]$
 $=\{0,0,0\}$



```

1. for  $j \leftarrow 1$  to  $n$ 
2.    $P[j] \leftarrow 0$ 
3. end for
4.  $\text{perm2}(n)$ 
  
```

过程 $\text{perm2}(m)$

```

1. if  $m=0$  then output  $P[1..n]$ 
2. else
3.   for  $j \leftarrow 1$  to  $n$ 
4.     if  $P[j]=0$  then
5.        $P[j] \leftarrow m$ 
6.        $\text{perm2}(m-1)$ 
7.        $P[j] \leftarrow 0$ 
8.     end if
9.   end for
10. end if
  
```

算法分析

- 由于存在有 $n!$ 个排列，过程 perm2 的第 1 步总共有 $nn!$ 个排列输出。
- 现在计算 for 循环的迭代次数
 - for 循环在每一个调用 perm2(m) 中被执行 n 次，再加上递归调用 perm2(m-1) 中执行的次数。
 - 当过程 perm2 以调用 perm2 (m)($m>0$) 形式调用时，数组 P 恰好包含 m 个 0，并且由于递归调用 perm2 (m-1) 将被执行恰好 m 次。因为在 $m=0$ 时，迭代次数是 0，所以迭代的次数可以用递推式表示如下：

$$f(m) = \begin{cases} 0 & \text{if } m=0 \\ mf(m-1)+n & \text{if } m \geq 1 \end{cases}$$

算法分析 (Continue)

令 $m!h(m)=f(m)$ (注意 $h(0)=0$). 那么 ,

$$m!h(m) = m(m-1)!h(m-1) + n$$

这个递推式的解是

$$h(m) = h(0) + \sum_{j=1}^m \frac{n}{j!} = n \sum_{j=1}^m \frac{1}{j!} < \sum_{j=1}^{\infty} \frac{1}{j!} < (e-1)$$

这里 $e=2.7182818\dots$

因此

$$f(m) = m!h(m) = nm! \sum_{j=1}^m \frac{1}{j!} < 2nm!$$

用 n 代入, 迭代的次数就变成

$$f(n) = nn! \sum_{j=1}^n \frac{1}{j!} < 2nn!$$

因此, 算法的运行时间是 $\square (nn!)$.

2.3.4 Radix sort (基数排序)

? 问题描述

- 令 $L=\{a_1, a_2, \dots, a_n\}$ 是一张有 n 个数的集合，其中每个数恰好有 k 位数字，就是说每个数具有 $d_k d_{k-1} \dots d_1$ 形式，这里 d_i 是 0 到 9 中的一个数字。
- 我们研究一种排序算法对 L 进行非降序排列，使算法在几乎所有的实际应用中都以线性时间运行。
- 在这个问题中，我们对整数的大小 k 施行归纳，而不是对数 n 这个对象来用归纳法。

方法一

- 根据它们的最高位把它们分类到 10 个表 $L_0, L_1, \dots, L_9$ 中去, 因此 $d_k = 0$ 的那些数就分到表 L_0 中, $d_k = 1$ 的那些数就分到表 L_1 中, $\dots$ 。
- 这一步完成后, 对于每个 L_i ($0 \leq i \leq 9$), 包含最高位是 i 的那些数。

? How to do next?

第一种选择...

- 用一种排序算法对每一张表排序，然后把排序得到的这些表连接成一张排序表。
- 在最坏的情况下，所有的数字可能有相同的最高位，这就意味着它们都在 1 张表中，而其余的 9 张表是空的。
- 因此如果表内排序用的算法需要 $O(n \log n)$ 时间，则这种方法的运行时间将是 $O(n \log n)$ 的。

第二种选择...

- 另一种可能性是对 d_{k-1} 位上每张表递归排序。
- 但这种方法将导致越来越多的附加表，这是很不方便的。

? Can we figure out more useful method?



惊奇的发现

- Given: $A = \{23, 32, 34, 45, 61, 78\}$

23 32 34 45 61 78

Sort A according to the **least** significant digit



惊奇的发现

- Given: $A = \{23, 32, 34, 45, 61, 78\}$

23 **32** **34** **45** **61** **78**

61 **32** **23** **34** **45** **78**

Sort A according to the **most** significant digit

Success!





Example

- Given: $A = \{7467, 1247, 3275, 6792, 9187, 9134, 4675, 1239\}$.

7467	6792	9134	9134	1239
1247	9134	1239	9187	1247
3275	3275	1247	1239	3275
6792	4675	7467	1247	4675
9187	7467	3275	3275	6792
9134	1247	4675	7467	7467
4675	9187	9187	4675	9134
1239	1239	6792	6792	9187

Success!

Radix sort (基数排序)



基本思想：

- 首先根据 d_1 位，把数分到 10 个表 $L_0, L_1, \dots, L_9$ 中，这样，那些 $d_1=0$ 的数分到表 L_0 中，那些 $d_1=1$ 的数分到表 L_1 中，等等。
- 接下来，表被用 $L_0, L_1, \dots, L_9$ 的顺序接起来。
- 再后来，把它们按照 d_2 分到 10 张表中，按顺序接起来。
- 再把它们按 d_k 分到表中并按顺序接起来后，所有的数就排好了序。



基数排序示意图

第1次收集

70	30	40	10	80	20	90	100	60	75	45
----	----	----	----	----	----	----	-----	----	----	----

第2次分配(按十位排序)

head[0] → 100

head[1] → 10

head[2] → 20

head[3] → 30

head[4] → 40 → 45

head[6] → 60

head[7] → 70 → 75

head[8] → 80

head[9] → 90

第2次收集

100	10	20	30	40	45	60	70	75	80	90
-----	----	----	----	----	----	----	----	----	----	----

第3次分配(按百位排序)

head[0] → 10 → 20 → 30 → 40 → 45 → 60 → 70 → 75 → 80 → 90

head[1] → 100

第3次收集

10	20	30	40	45	60	70	75	80	90	100
----	----	----	----	----	----	----	----	----	----	-----

算法 2.3.4 RADIXSORT

输入：一张有 n 个数的表 $L=\{a_1...a_n\}$ 和 k 位数字。

输出：按非降序排列的 L 。

1. **for** $j \leftarrow 1$ **to** k
2. 准备 10 个空表 $L_0, L_1, \dots, L_9$.
3. **while** L 非空
4. $a \leftarrow L$ 中的下一元素；删除 L 中的 a
5. $i \leftarrow a$ 中的第 j 位数字；将 a 加入表 L_i 中
6. **end while**
7. $L \leftarrow L_0$
8. **for** $i \leftarrow 1$ **to** 9
9. $L \leftarrow L, L_i$ { 将表 L_i 加入表 L 中 }
10. **end for**
11. **end for**
12. **return** L

算法分析

- 根据某一位数字把数分到表中去的方法做了 k 遍，每一遍的代价是 $\Theta(n)$ 时间，这样算法的时间是 $\Theta(kn)$ 。如果 k 是常数，运行时间就是 $\Theta(n)$ 。
- 算法用了 $\Theta(n)$ 空间，因为需要 10 个表，这些表全部的大小是 $\Theta(n)$ 。

2.3.5 Finding the majority element (寻找多数元素)

? 问题描述

- 令 $A[1..n]$ 是一个整数序列， A 中的整数 a 如果在 A 中出现的次数多于 $\lfloor n/2 \rfloor$ ，那么 a 称为多数元素。
- 例如，在序列 1,3,2,3,3,4,3 中，3 是多数元素，因为 7 个元素中它出现 4 次。

方法 1(brute-force method)

- 基本思想

- 蛮力方法是把每个元素和其他每个元素比较，并且对每个元素计数。
- 如果某个元素的计数大于 $\lfloor n/2 \rfloor$ ，就可以断言它是多数元素；否则，在序列中就没有多数元素。
- 但是这样的比较次数是

$$n(n-1)/2 = \Theta(n^2)$$



方法 2

- 基本思想

- 比较有效的算法是对这些元素排序，并且计算每个元素在序列中出现多少次。
- 这在最坏情况下的代价是 $O(n \log n)$ 。因为在最坏情况下，排序这一步需要 $O(n \log n)$ 次比较 (定理 12.2)。

方法 3

- 基本思想

- 寻找中间元素，就是第 $\lfloor n/2 \rfloor$ 元素。因为多数元素，在排序的序列中一定是中间元素，可以扫描这个序列来测试中间元素是否确实是多数元素。由于中间元素可以在 $O(n)$ 时间内找到，这个方法要花费 $O(n)$ 时间。
- 看到在中项寻找算法的时间复杂性中隐藏的常数太大，并且算法是相当复杂的。



Is there much more efficient method?



更有效算法



观察结论 2.1

- 在原序列中去除两个不同的元素后，那么在原序列中的多数元素在新序列中还是多数元素。
- 这个观察结论支持下述寻找多数元素候选者的过程。



基本思想

将计数器置 1，并令 $c=A[1]$

- 从 $A[2]$ 开始，逐个地扫描元素，如果被扫描的元素和 c 相等，则计数器加 1；如果元素不等于 c ，则计数器减 1；
- 如果所有的元素都已经扫描完毕并且计数器大于 0，那么返回。作为多数元素的候选者。
- 如果在 c 和 $A[j](1<j<n)$ 比较时计数器为 0，那么对于 $A[j+1..n]$ 上的元素递归调用 candidate 过程。
- 注意减少计数器就是观察结论 2.1 中所述去除两个不同元素的思想的实现。

算法 2.3.5 MAJORITY

Input: n 个元素的数组 $A[1..n]$

Output: 若存在则输出多数元素，；否则输出 none。

1. $c \leftarrow \text{candidate}(1)$ // 获取候选者
2. $\text{count} \leftarrow 0$
3. **for** $j \leftarrow 1$ **to** n
4. **if** $A[j]=c$ **then** $\text{count} \leftarrow \text{count}+1$ // 候选者进行计数
2. **end for**
6. **if** $\text{count} > \lfloor n/2 \rfloor$ **then return** c
7. **else return** none

过程 $\text{candidate}(m)$

1. $j \leftarrow m; c \leftarrow A[m]; \text{count} \leftarrow 1$
2. **while** $j < n$ **and** $\text{count} > 0$ // 当 $\text{count} == 0$ 时，循环退出，相当于除去 2 个不同元素。
3. $j \leftarrow j+1$
4. **if** $A[j]=c$ **then** $\text{count} \leftarrow \text{count}+1$
2. **else** $\text{count} \leftarrow \text{count}-1$
6. **end while**
7. **if** $j=n$ **then return** c // 获取候选者
8. **else return** $\text{candidate}(j+1)$

2.3.6 汉诺塔

- 大梵天创造世界的时候做了三根金刚石柱子，在一根柱子上从下往上按照大小顺序摞着 64 片黄金圆盘。大梵天命令婆罗门把圆盘从下面开始按大小顺序重新摆放在另一根柱子上。并且规定，在小圆盘上不能放大圆盘，在三根柱子之间一次只能移动一个圆盘。



操作示例

- 只有一个圆盘，直接从 $a \rightarrow c$
- 如果两个圆盘， $a \rightarrow b, a \rightarrow c, b \rightarrow c$
- 如果三个圆盘， $a \rightarrow c, a \rightarrow b, c \rightarrow b, a \rightarrow c, b \rightarrow a, b \rightarrow c, a \rightarrow c$
- 如果四个圆盘：

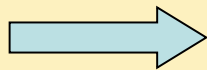


算法分析

- 假设一共有 n 个盘子
- 首先借助于 c 将 $n-1$ 个盘子从 $a \rightarrow b$
- 最后将第 n 个盘子从 $a \rightarrow c$
- 此时，问题是将 b 上的 $n-1$ 个盘子移动到 c

Hanoi(n , x , y ,

z);



Hanoi($n-1$, x , z , y);

move(n , x , z): 将第 n 个圆盘从 x 移到 z ;

Hanoi($n-1$, y , x , z)

“大问题”转化为若干个“小问题”求解

算法分析

- 设移动 n 个盘子的代价是 $C(n)$
- 将一个盘子从 a 移动到 c 的代价是 1
- $C(n)=2C(n-1)+1$
- $C(n)=2(2C(n-2)+1)=2^2C(n-2)+2$
 $=2^3C(n-3)+2^2+2$
 $= \dots$
 $=2^{n-1}+2^{n-2}+\dots+2$
 $=2^n-1$



2.3.7 取棋子游戏

- 有 $2n$ 个棋子 ($n \geq 4$) 排成一行, 开始位置为白子全部在左边, 黑子全部在右边, 如下图为 $n=5$ 的情形:

○ ○ ○ ○ ○ ● ● ● ● ●

- 移动棋子的规则是: 每次必须同时移动相邻的两个棋子, 颜色不限, 可以左移也可以右移到空位上去, 但不能调换两个棋子的左右位置。每次移动必须跳过若干个棋子 (不能平移), 要求最后能移成黑白相间的一行棋子。如 $n=5$ 时, 成为:
- ● ○ ● ○ ● ○ ● ○ ●

操作示例

○ ○ ○ ○ ○ ● ● ● ● ●

○ ○ ○ ○ ● ● ● ● ○ ●

○ ○ ○ ● ● ● ○ ● ○ ●

○ ○ ● ● ○ ● ○ ● ○ ●

○ ● ○ ● ○ ● ○ ● ○ ●



算法分析

- 假设一共有 $2n$ 个棋子
- 第一步，将中间的一黑一白两个棋子移动到后
- 此时不满足要求的有 $2(n-1)$ 个棋子，其中前 $n-1$ 个是白色棋子，从 n 到 $2n-2$ 是黑色棋子；
- 可以重复第一步的操作
- 总共的移动次数是 $2(n-1)$ 次





Exercises

- 2.7
- 2.12
- 2.29
- 2.31
- 一个数列的首项 $a_1=0$, 后续奇数项和偶数项的计算公式分别为 $a_{2n}=a_{2n-1}+2$, $a_{2n+1}=a_{2n-1}+a_{2n}-1$, 设计一个递归算法求数列第 n 项。